

Final Assignment Submission Guide & Rubric

Advanced Database Systems (CS-402)

Faculty of Information Technology

Academic Year 2025–2026

1 Assignment Overview

The final project is a group effort (2–3 students) to design and implement a robust software system using **Polyglot Persistence**. Students must strategically utilize **Oracle/MS SQL Server** for transactional data (ACID) and **MongoDB** for flexible content and analytical logging (BASE).

2 Hand-in Requirements

Each group must submit a digital archive (ZIP) containing:

2.1 1. Technical Implementation (Source Code)

- Functional application code and database initialization scripts (.sql and .js).
- A README.md explaining how to run the system.

2.2 2. Final Project Report (PDF Structure)

The report must contain a **Group Section** (General Design) and an **Individual Section** for each member:

1. **[Group] Business Analysis:** Scenario description and User Requirements.
2. **[Group] System Architecture:** Global diagram of the Polyglot interaction.
3. **[Group] Database Design:** Full ERD and NoSQL Collection structures.
4. **[Individual] Technical Defense (1–2 pages per student):**
 - **Ownership:** Clearly list which modules/features you authored.
 - **Justification:** Defend your design choices (e.g., "I chose Repeatable Read for the loan table because...").
 - **Logic Walkthrough:** A snippet of your most complex code (Trigger/CTE/Aggregation) with a step-by-step explanation of its execution logic.
 - **AI Critique:** Identify one specific instance where you had to manually correct or optimize an AI-generated database pattern.
5. **[Appendix] AI Audit Log:** Full list of prompts used by the team.

3 Evaluation Rubric: Group vs. Individual

Total Grade = (Group Grade $\times$ 0.85) + (Individual Defense $\times$ 0.15).

Category	Weight	Type	Excellence Criteria (9.0 – 10.0)
Design & Architecture	35%	Group	Optimal split between SQL/NoSQL. Correct use of Normalization vs. Embedding.
Technical Execution	35%	Group	Functional implementation of Triggers, Recursive CTEs, and Sagas/2PC.
Report Quality	15%	Group	Clear diagrams, professional flow, and transparent AI prompt documentation.
Individual Defense	15%	Indiv.	Student proves ownership of their code. Explains "Why" and can modify logic live.

3.1 Individual Performance Modifiers (Redlines)

The individual score is derived from the "Technical Defense" section and the Q&A. Failure in these areas will result in an individual grade lower than the group grade:

- **Architectural Argumentation:** Can the student justify the choice of ACID vs. BASE for their module?
- **Logic Ownership:** Can the student explain the $O(N)$ complexity or the concurrency risks of their code?
- **AI Critical Review:** Can the student demonstrate how they verified AI output for correctness?

4 Special Instructions on AI Usage

AI is a powerful co-pilot, but the pilot (the student) must know how the engine works.

The "Modify-on-the-Fly" Test

During the oral defense, I will randomly select one logic block from your **Individual Technical Defense** and ask you to modify it. Examples:

- "You used a `RANK()` function here. Change it to `ROW_NUMBER()` live and tell me how the business result changes."
- "You implemented Pessimistic Locking. If I remove `FOR UPDATE`, what specific race condition occurs in your scenario?"

Failure to explain or modify your own code will be interpreted as a lack of ownership.

– Final Project Hand-in Requirements finalized –